

Sistema de Gestão de Territórios e Publicações (GTP)

Documento 20 – Histórico de Versões e Roadmap

Projeto: Sistema de Gestão de Territórios e Publicações (GTP)

Versão: 1.0.0

Status: Em Elaboração

Data: Julho/2026

Autor: Fabio André Zatta

1 – Introdução

1.1 Objetivo

Este documento tem como objetivo registrar a evolução histórica do **Gestor de Territórios e Publicações (GTP)**, documentando as versões disponibilizadas, as funcionalidades implementadas, as correções realizadas, as mudanças arquiteturais e o planejamento das futuras evoluções da plataforma.

Além de servir como registro oficial das entregas do projeto, este documento estabelece um roadmap estratégico para orientar o crescimento do sistema de forma organizada, transparente e alinhada às necessidades dos usuários e da equipe de desenvolvimento.

1.2 Escopo

Este documento contempla:

- histórico das versões do sistema;
- política de versionamento;
- changelog das funcionalidades;
- registro de correções;
- mudanças arquiteturais;
- funcionalidades planejadas;
- roadmap tecnológico;
- estratégia de evolução do produto;
- planejamento de releases futuras.

O documento abrange tanto as versões de desenvolvimento quanto as versões destinadas aos ambientes de homologação e produção.

1.3 Público-Alvo

Este documento destina-se a:

- Desenvolvedores;
- Arquitetos de Software;

- Product Owner;
- Administradores do Sistema;
- DevOps;
- Equipe de Suporte;
- Gestores do Projeto;
- Colaboradores responsáveis pela evolução do GTP.

1.4 Objetivos do Histórico de Versões

O histórico de versões possui os seguintes objetivos:

- registrar a evolução da plataforma;
- documentar alterações realizadas;
- facilitar auditorias técnicas;
- melhorar a rastreabilidade das mudanças;
- apoiar o planejamento das próximas versões;
- facilitar a comunicação entre as equipes;
- preservar o conhecimento do projeto.

1.5 Visão Geral da Evolução do Produto

O desenvolvimento do GTP é organizado de forma incremental.

Cada versão representa um conjunto de melhorias planejadas e validadas antes da disponibilização aos usuários.

A evolução do sistema contempla:

- novas funcionalidades;
- melhorias de desempenho;
- evolução arquitetural;
- correções de defeitos;
- aprimoramentos de segurança;
- melhorias na experiência do usuário;

- evolução da infraestrutura.

Essa abordagem permite entregas contínuas com menor risco operacional.

1.6 Estratégia de Versionamento

O GTP adota o padrão **Versionamento Semântico (Semantic Versioning – SemVer)**.

A estrutura da versão segue o formato:

MAJOR.MINOR.PATCH

Onde:

Elemento Significado

MAJOR Alterações incompatíveis ou grandes evoluções

MINOR Novas funcionalidades compatíveis

PATCH Correções e pequenos ajustes

Exemplos:

- 1.0.0
- 1.1.0
- 1.2.3
- 2.0.0

Essa estratégia facilita o controle da evolução da plataforma.

1.7 Política de Releases

As versões podem ser classificadas em diferentes categorias.

Tipo	Finalidade
Alpha	Desenvolvimento inicial
Beta	Testes com funcionalidades completas
Release Candidate (RC)	Versão candidata à produção

Tipo	Finalidade
Stable	Versão oficial de produção
Hotfix	Correções urgentes

Cada categoria possui objetivos específicos dentro do ciclo de desenvolvimento.

1.8 Identificação das Versões

Cada release deve possuir informações mínimas.

Exemplo:

Campo	Conteúdo
Versão	1.0.0
Data da publicação	DD/MM/AAAA
Ambiente	Produção
Responsável	Equipe de Desenvolvimento
Tipo	Stable
Observações	Primeira versão oficial

Esses registros garantem rastreabilidade e histórico completo das implantações.

1.9 Convenções

Neste documento serão utilizados:

- tabelas de versões;
- registros de mudanças;
- cronogramas de releases;
- roadmaps;
- histórico de funcionalidades;
- matrizes de evolução;
- registros de decisões arquiteturais.

Toda alteração significativa no GTP deverá ser registrada neste documento para garantir a rastreabilidade da evolução do projeto.

1.10 Conclusão do Capítulo

Este capítulo apresentou os objetivos, o escopo e a estrutura do **Documento 20 – Histórico de Versões e Roadmap**, estabelecendo as diretrizes para o registro da evolução do GTP e para o planejamento das futuras versões da plataforma.

Ao adotar uma estratégia de versionamento semântico e um roadmap estruturado, o projeto ganha maior previsibilidade, transparência e capacidade de planejamento.

2 – Histórico das Versões

2.1 Objetivo

Este capítulo apresenta o histórico oficial das versões do **Gestor de Territórios e Publicações (GTP)**, registrando a evolução funcional, técnica e arquitetural da plataforma desde sua concepção até as versões futuras planejadas.

O objetivo é garantir a rastreabilidade das mudanças, facilitar auditorias, apoiar processos de manutenção e fornecer uma visão clara da evolução do sistema ao longo do tempo.

2.2 Visão Geral da Evolução

O desenvolvimento do GTP foi estruturado em ciclos incrementais, permitindo a entrega contínua de funcionalidades, melhorias e correções.

Cada versão representa um marco na evolução do projeto e contempla:

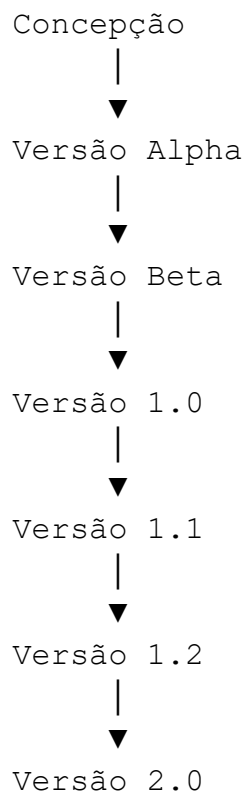
- novas funcionalidades;
- melhorias de desempenho;
- correções de defeitos;
- atualizações tecnológicas;
- evolução da arquitetura;
- aprimoramentos de segurança;

- otimizações de infraestrutura.

Essa abordagem reduz riscos e facilita a evolução sustentável da plataforma.

2.3 Linha do Tempo das Versões

A evolução do projeto pode ser representada conforme a linha do tempo abaixo.



Novas versões serão incorporadas conforme a evolução do sistema.

2.4 Registro das Versões

Cada versão deve possuir um registro padronizado.

Modelo recomendado:

Campo	Descrição
Versão	Identificador da release
Data	Data da publicação
Tipo	Alpha, Beta, RC, Stable ou Hotfix

Campo	Descrição
Ambiente	Desenvolvimento, Homologação ou Produção
Responsável	Equipe responsável
Objetivo	Finalidade da versão
Observações	Informações adicionais

Esse modelo facilita consultas e auditorias futuras.

2.5 Histórico Inicial do Projeto

Versão 0.1.0 – Concepção

Objetivo

Início da concepção do projeto e definição dos requisitos iniciais.

Principais atividades:

- levantamento de requisitos;
- definição do escopo;
- estudo da arquitetura;
- escolha das tecnologias;
- elaboração da documentação inicial.

Versão 0.5.0 – Protótipo

Objetivo

Construção do primeiro protótipo funcional.

Principais entregas:

- estrutura inicial do backend;
- estrutura inicial do frontend;
- autenticação básica;
- organização dos módulos;
- primeiros testes técnicos.

Versão 0.9.0 – Beta

Objetivo

Disponibilização para testes internos.

Principais entregas:

- funcionalidades essenciais implementadas;
- banco de dados estruturado;
- API REST funcional;
- interface principal concluída;
- validação dos fluxos de negócio.

Versão 1.0.0 – Primeira Release Oficial

Objetivo

Primeira versão estável para utilização em produção.

Principais entregas:

- autenticação completa;
- gerenciamento de usuários;
- gerenciamento de territórios;
- gerenciamento de publicações;
- auditoria;
- segurança baseada em JWT;
- documentação oficial.

Essa versão representa o primeiro marco oficial do GTP.

2.6 Evolução das Funcionalidades

Ao longo das versões, diversas funcionalidades foram incorporadas.

Área	Evolução
-------------	-----------------

Autenticação	Login, JWT, perfis e permissões
--------------	---------------------------------

Área	Evolução
Usuários	Cadastro, edição e administração
Territórios	Cadastro, distribuição e controle
Publicações	Controle completo de publicações
Administração	Configurações e parametrizações
Segurança	Auditoria, logs e autenticação
Relatórios	Indicadores e consultas
Infraestrutura	Docker, CI/CD e monitoramento

Cada evolução deve ser registrada na versão correspondente.

2.7 Evolução Tecnológica

A arquitetura também evolui ao longo das versões.

Componente	Evolução
Java	Atualizações LTS
Spring Boot	Evolução das versões estáveis
React	Atualizações da interface
PostgreSQL	Evolução da persistência
Docker	Modernização da infraestrutura
Flyway	Evolução das migrações
Spring Security	Aprimoramento da segurança

Todas as mudanças tecnológicas devem ser registradas.

2.8 Correções Importantes

Além das novas funcionalidades, cada versão deve registrar as correções implementadas.

Exemplos:

- correções de autenticação;
- ajustes de desempenho;
- otimizações das consultas SQL;
- correções de interface;
- melhorias de segurança;
- correção de vulnerabilidades;
- ajustes em integrações.

Esse histórico facilita futuras investigações técnicas.

2.9 Mudanças de Arquitetura

Alterações estruturais relevantes também devem ser documentadas.

Exemplos:

- reorganização dos módulos;
- alteração da estrutura de pacotes;
- adoção de novos padrões arquiteturais;
- atualização das dependências principais;
- mudanças na infraestrutura;
- adoção de novos mecanismos de autenticação;
- evolução da estratégia de deploy.

Essas informações auxiliam na compreensão da evolução técnica do sistema.

2.10 Registro de Hotfixes

Correções emergenciais devem ser registradas separadamente.

Modelo sugerido:

Versão	Data	Problema	Solução
1.0.1	DD/MM/AAAA	Correção crítica	Ajuste aplicado

Versão	Data	Problema	Solução
1.0.2	DD/MM/AAAA	Correção de segurança	Atualização realizada

Essa prática melhora a rastreabilidade das intervenções emergenciais.

2.11 Histórico de Releases

A tabela abaixo resume o histórico geral das versões.

Versão	Tipo	Situação
0.1.0	Alpha	Planejamento inicial
0.5.0	Alpha	Protótipo
0.9.0	Beta	Testes internos
1.0.0	Stable	Primeira versão oficial
1.1.0	Planejada	Evolução funcional
2.0.0	Planejada	Nova geração da plataforma

Novas linhas devem ser adicionadas a cada release.

2.12 Critérios para Registro de Versões

Cada nova versão deve registrar:

- data da publicação;
- número da versão;
- tipo da release;
- funcionalidades implementadas;
- correções realizadas;
- alterações arquiteturais;
- atualizações tecnológicas;
- problemas conhecidos;
- responsáveis pela entrega.

Esse padrão garante consistência entre todas as releases.

2.13 Boas Práticas

Para manter um histórico confiável, recomenda-se:

- registrar todas as releases;
- utilizar Versionamento Semântico (SemVer);
- manter o changelog atualizado;
- documentar hotfixes;
- registrar decisões arquiteturais relevantes;
- vincular versões aos artefatos de implantação;
- manter rastreabilidade entre requisitos, código e documentação.

2.14 Conclusão do Capítulo

Este capítulo apresentou o histórico das versões do GTP, estabelecendo uma estrutura padronizada para registrar a evolução funcional, técnica e arquitetural da plataforma.

A manutenção desse histórico fortalece a governança do projeto, facilita auditorias, apoia atividades de manutenção e fornece uma visão clara da evolução do sistema ao longo do tempo.

3 – Changelog

3.1 Objetivo

Este capítulo apresenta o **Changelog Oficial** do **Gestor de Territórios e Publicações (GTP)**, registrando de forma cronológica todas as alterações realizadas em cada versão da plataforma.

O objetivo é manter um histórico detalhado das funcionalidades implementadas, correções, melhorias técnicas, alterações arquiteturais, atualizações de segurança e modificações na infraestrutura, garantindo rastreabilidade e transparência durante todo o ciclo de vida do projeto.

3.2 O que é um Changelog

O changelog é um registro estruturado das alterações realizadas entre uma versão e outra do sistema.

Cada entrada deve documentar:

- funcionalidades adicionadas;
- funcionalidades alteradas;
- funcionalidades removidas;
- correções de defeitos;
- melhorias de desempenho;
- atualizações de segurança;
- mudanças na infraestrutura;
- alterações arquiteturais.

Esse histórico facilita a comunicação entre as equipes e o acompanhamento da evolução da plataforma.

3.3 Estrutura do Changelog

Cada versão deve seguir um padrão único de documentação.

Modelo recomendado:

Campo	Descrição
Versão	Identificador da release
Data	Data de publicação
Tipo	Alpha, Beta, RC, Stable ou Hotfix
Responsável	Equipe responsável
Ambiente	Desenvolvimento, Homologação ou Produção
Resumo	Descrição geral da release

Após essas informações, as alterações devem ser organizadas por categoria.

3.4 Categorias de Alterações

O GTP utiliza as seguintes categorias para organizar o changelog.

Categoria	Descrição
Added	Novas funcionalidades
Changed	Alterações em funcionalidades existentes
Fixed	Correções de defeitos
Removed	Funcionalidades removidas
Security	Atualizações de segurança
Performance	Melhorias de desempenho
Infrastructure	Alterações de infraestrutura
Documentation	Atualizações da documentação

Essa organização segue boas práticas amplamente adotadas em projetos de software.

3.5 Modelo de Registro

Exemplo de estrutura para uma versão:

Versão 1.0.0

Data: DD/MM/AAAA

Added

- Cadastro de usuários
- Cadastro de territórios
- Cadastro de publicações

Changed

- Melhorias na autenticação

Fixed

- Correção de login

Security

- Implementação de JWT

Performance

- Otimização das consultas

Documentation

- Inclusão do Manual do Usuário

Esse formato padroniza o registro das alterações.

3.6 Exemplo — Versão 0.1.0

Alpha

Added

- criação inicial do projeto;
- definição da arquitetura;
- configuração do repositório;
- estrutura inicial do backend;
- estrutura inicial do frontend.

Documentation

- documentação inicial;
- levantamento de requisitos.

3.7 Exemplo — Versão 0.5.0

Protótipo

Added

- autenticação inicial;
- estrutura do banco de dados;
- primeiras APIs REST;
- layout inicial do frontend.

Changed

- reorganização da arquitetura.

Documentation

- atualização da documentação técnica.

3.8 Exemplo — Versão 0.9.0

Beta

Added

- gerenciamento de usuários;
- gerenciamento de territórios;
- gerenciamento de publicações;
- controle de permissões.

Fixed

- correções de autenticação;
- ajustes na API;
- correções de interface.

Performance

- otimização das consultas SQL.

3.9 Exemplo — Versão 1.0.0

Primeira Release Oficial

Added

- autenticação completa;
- controle de perfis;
- auditoria;
- dashboard;
- gerenciamento administrativo;

- relatórios básicos.

Security

- autenticação JWT;
- criptografia de senhas;
- controle de permissões.

Infrastructure

- Docker;
- Docker Compose;
- Flyway;
- PostgreSQL.

Documentation

- documentação completa da versão 1.0.

3.10 Registro de Correções

Correções devem ser documentadas de forma objetiva.

Modelo recomendado:

Versão	Tipo	Correção
---------------	-------------	-----------------

1.0.1	Hotfix	Correção da autenticação
-------	--------	--------------------------

1.0.2	Hotfix	Correção de desempenho
-------	--------	------------------------

1.0.3	Hotfix	Ajuste de segurança
-------	--------	---------------------

Esse padrão facilita consultas futuras.

3.11 Registro de Segurança

Atualizações de segurança merecem destaque específico.

Devem incluir:

- vulnerabilidade corrigida;
- impacto;

- versão afetada;
- versão corrigida;
- data da atualização;
- recomendações adicionais.

Essa prática fortalece a governança da segurança da informação.

3.12 Registro de Mudanças Arquiteturais

Alterações significativas na arquitetura devem conter:

- motivo da mudança;
- componentes afetados;
- impacto esperado;
- benefícios;
- riscos;
- documentação relacionada.

Essas informações auxiliam futuras decisões técnicas.

3.13 Integração com Git

O changelog deve manter correspondência com o histórico do repositório Git.

Recomenda-se relacionar cada release com:

- tag da versão;
- branch utilizada;
- pull requests incorporados;
- commits relevantes;
- pipeline de CI/CD correspondente.

Essa integração melhora a rastreabilidade entre código-fonte e documentação.

3.14 Boas Práticas

Para manter um changelog consistente, recomenda-se:

- atualizar o registro a cada release;
- utilizar linguagem objetiva;
- organizar alterações por categoria;
- documentar apenas mudanças relevantes;
- manter consistência entre documentação e código;
- registrar hotfixes separadamente;
- preservar o histórico completo das versões.

3.15 Exemplo de Estrutura Geral

```
Versão
|
|— Added
|— Changed
|— Fixed
|— Removed
|— Security
|— Performance
|— Infrastructure
|— Documentation
```

Essa estrutura deve ser adotada para todas as futuras versões do GTP.

3.16 Conclusão do Capítulo

Este capítulo apresentou o modelo oficial de changelog do GTP, estabelecendo um padrão para registrar todas as alterações realizadas ao longo do ciclo de vida da plataforma.

A manutenção desse histórico fortalece a rastreabilidade das mudanças, facilita auditorias, melhora a comunicação entre as equipes e apoia a evolução contínua do sistema.

4 – Evolução Arquitetural

4.1 Objetivo

Este capítulo documenta a evolução arquitetural do **Gestor de Territórios e Publicações (GTP)**, registrando as principais decisões técnicas adotadas ao longo do desenvolvimento da plataforma.

Seu objetivo é preservar o histórico da arquitetura, justificar mudanças estruturais, facilitar futuras manutenções e fornecer uma base para a evolução contínua do sistema.

4.2 Visão Geral da Evolução

A arquitetura do GTP foi concebida para ser moderna, modular, escalável e de fácil manutenção, evoluindo gradualmente conforme o crescimento do projeto.

A evolução arquitetural considera:

- organização modular;
- separação de responsabilidades;
- desacoplamento entre frontend e backend;
- padronização da comunicação via API REST;
- adoção de boas práticas de segurança;
- automação de infraestrutura;
- preparação para escalabilidade.

Essa abordagem permite que novas funcionalidades sejam incorporadas sem comprometer a estabilidade do sistema.

4.3 Linha do Tempo da Arquitetura

A evolução da arquitetura pode ser representada da seguinte forma:

Protótipo Inicial



Arquitetura Monolítica Modular



API REST Padronizada



Autenticação JWT



Dockerização



CI/CD



Observabilidade



Arquitetura Preparada para Escalabilidade

Cada etapa representa um avanço na maturidade técnica da plataforma.

4.4 Evolução do Backend

O backend passou por diversas melhorias estruturais.

Primeira fase

- organização inicial dos pacotes;
- implementação das entidades;
- configuração do Spring Boot;
- persistência com PostgreSQL.

Segunda fase

- separação em camadas;
- criação de serviços;
- padronização dos controladores REST;
- implementação de DTOs;

- utilização do MapStruct.

Terceira fase

- autenticação JWT;
- Spring Security;
- tratamento global de exceções;
- auditoria;
- validações com Jakarta Validation.

Evolução futura

- arquitetura orientada a eventos;
- cache distribuído;
- mensageria assíncrona;
- APIs versionadas.

4.5 Evolução do Frontend

A interface também passou por evolução contínua.

Primeira versão

- React;
- Vite;
- React Router;
- estrutura inicial das páginas.

Segunda versão

- componentes reutilizáveis;
- Context API;
- autenticação integrada;
- layouts responsivos.

Terceira versão

- dashboards;

- melhorias de UX;
- otimizações de desempenho;
- padronização visual.

Evolução planejada

- design system próprio;
- biblioteca de componentes compartilhados;
- internacionalização (i18n);
- Progressive Web App (PWA);
- acessibilidade ampliada conforme WCAG.

4.6 Evolução do Banco de Dados

A camada de persistência também evoluiu.

Principais melhorias:

- modelagem relacional refinada;
- normalização das tabelas;
- índices de desempenho;
- migrações com Flyway;
- auditoria dos dados;
- otimização das consultas.

Planejamento futuro:

- particionamento de tabelas de grande volume, quando necessário;
- otimização contínua de índices;
- replicação para cenários de alta disponibilidade, caso a infraestrutura exija.

4.7 Evolução da Segurança

A arquitetura de segurança evoluiu significativamente.

Evolução	Objetivo
Autenticação JWT	Controle de acesso
Spring Security	Proteção das APIs
BCrypt	Criptografia de senhas
Controle de Perfis	Autorização baseada em papéis
Auditoria	Rastreabilidade
HTTPS	Comunicação segura

Evoluções futuras incluem autenticação multifator (MFA), quando aplicável, políticas mais avançadas de gerenciamento de sessões e ampliação dos mecanismos de monitoramento de segurança.

4.8 Evolução da Infraestrutura

A infraestrutura foi preparada para ambientes modernos.

Principais marcos:

- Docker;
- Docker Compose;
- ambientes separados;
- CI/CD;
- monitoramento;
- logs centralizados;
- backups automatizados.

Planejamento futuro:

- Kubernetes;
- balanceamento de carga;
- autoescalabilidade;
- observabilidade avançada;
- infraestrutura como código (IaC).

4.9 Decisões Arquiteturais (ADR)

As principais decisões arquiteturais devem ser registradas por meio de **Architecture Decision Records (ADR)**.

Modelo sugerido:

Campo	Conteúdo
Identificador	ADR-001
Data	DD/MM/AAAA
Contexto	Problema identificado
Decisão	Solução adotada
Consequências	Benefícios e impactos
Status	Ativa, Revisada ou Substituída

O uso de ADRs preserva o contexto técnico das decisões tomadas durante a evolução do projeto.

4.10 Marcos Arquiteturais

Os principais marcos da evolução do GTP são:

Marco	Descrição
Arquitetura inicial	Estrutura básica do sistema
Modularização	Separação por responsabilidades
API REST	Comunicação padronizada
Segurança	Autenticação e autorização
Dockerização	Padronização dos ambientes
CI/CD	Automatização das implantações
Observabilidade	Monitoramento e logs
Escalabilidade	Preparação para crescimento

Esses marcos representam a maturidade progressiva da arquitetura.

4.11 Critérios para Evolução

Toda alteração arquitetural deve considerar:

- impacto funcional;
- desempenho;
- segurança;
- escalabilidade;
- manutenibilidade;
- compatibilidade;
- facilidade de testes;
- custo de implementação.

Mudanças significativas devem ser avaliadas e aprovadas antes da implementação.

4.12 Integração com os Demais Documentos

A evolução arquitetural está diretamente relacionada aos seguintes documentos:

Documento	Relação
Documento 05 – Arquitetura Geral	Visão arquitetural da solução
Documento 06 – Arquitetura Backend	Evolução do backend
Documento 07 – Arquitetura Frontend	Evolução da interface
Documento 10 – Segurança	Evolução da segurança
Documento 11 – Banco de Dados	Evolução da persistência
Documento 14 – Deploy e Infraestrutura	Evolução da infraestrutura

Essa integração garante consistência entre arquitetura, implementação e documentação.

4.13 Boas Práticas

Para conduzir a evolução arquitetural do GTP, recomenda-se:

- documentar todas as decisões relevantes;
- utilizar ADRs para mudanças estruturais;
- evitar acoplamento excessivo;
- manter alta coesão entre os módulos;
- revisar periodicamente a arquitetura;
- adotar padrões consolidados pela comunidade;
- avaliar continuamente riscos técnicos (*Technical Debt*);
- alinhar a evolução tecnológica aos objetivos do projeto.

4.14 Conclusão do Capítulo

Este capítulo apresentou a evolução arquitetural do GTP, descrevendo a transformação da plataforma desde sua estrutura inicial até uma arquitetura moderna, modular e preparada para crescimento.

O registro sistemático das decisões arquiteturais fortalece a governança técnica do projeto, facilita a integração de novos desenvolvedores e fornece uma base sólida para futuras evoluções.

5 – Roadmap de Funcionalidades

5.1 Objetivo

Este capítulo apresenta o **Roadmap de Funcionalidades** do **Gestor de Territórios e Publicações (GTP)**, estabelecendo a visão estratégica para a evolução funcional da plataforma.

O roadmap organiza as funcionalidades previstas em diferentes horizontes de planejamento, permitindo que a evolução do sistema ocorra de forma estruturada, incremental e alinhada às necessidades dos usuários e aos objetivos do projeto.

5.2 Visão Estratégica

O desenvolvimento do GTP segue uma estratégia de evolução contínua, priorizando:

- estabilidade da plataforma;
- segurança da informação;
- melhoria da experiência do usuário;
- automação de processos;
- aumento da produtividade;
- escalabilidade;
- facilidade de manutenção.

As funcionalidades são planejadas considerando seu valor para os usuários e o impacto técnico de sua implementação.

5.3 Critérios de Priorização

A definição das prioridades considera os seguintes critérios:

Critério	Objetivo
Valor para o usuário	Benefício direto ao usuário final
Impacto operacional	Melhoria nos processos da congregação
Complexidade técnica	Esforço estimado de desenvolvimento
Dependências	Relação com outras funcionalidades
Segurança	Necessidade de proteção dos dados
Desempenho	Impacto na performance do sistema
Conformidade	Atendimento aos requisitos definidos

As funcionalidades de maior valor e menor complexidade tendem a ser implementadas primeiro.

5.4 Horizonte de Planejamento

O roadmap é dividido em três horizontes principais.

Horizonte Período

Curto prazo Próximas versões

Médio prazo Evoluções subsequentes

Longo prazo Expansões estratégicas

Essa divisão facilita o planejamento e a revisão periódica das prioridades.

5.5 Roadmap de Curto Prazo

As funcionalidades previstas para as próximas versões concentram-se na consolidação da plataforma.

Funcionalidade	Objetivo
Aperfeiçoamento do gerenciamento de territórios	Melhorar o controle e distribuição
Melhorias no gerenciamento de publicações	Simplificar operações
Dashboards aprimorados	Ampliar indicadores operacionais
Relatórios avançados	Facilitar análises
Otimizações de desempenho	Reduzir tempos de resposta
Melhorias de usabilidade	Tornar a interface mais intuitiva
Ajustes de segurança	Reforçar a proteção da plataforma

Essas entregas priorizam estabilidade e experiência do usuário.

5.6 Roadmap de Médio Prazo

Após a consolidação da versão inicial, o projeto poderá evoluir com novas funcionalidades.

Exemplos:

Funcionalidade	Benefício
Agenda de atividades	Organização das atividades da congregação
Calendário integrado	Planejamento operacional
Painéis analíticos avançados	Apoio à tomada de decisão
Exportação ampliada de relatórios	Compartilhamento de informações
Configurações avançadas	Maior flexibilidade administrativa
Notificações inteligentes	Comunicação mais eficiente
Histórico detalhado de operações	Maior rastreabilidade

Essas funcionalidades ampliam o valor da plataforma sem alterar sua arquitetura principal.

5.7 Roadmap de Longo Prazo

As evoluções estratégicas incluem funcionalidades de maior abrangência.

Possibilidades futuras:

Funcionalidade	Objetivo
Aplicativo móvel	Acesso em dispositivos móveis
Progressive Web App (PWA)	Melhor experiência offline
Modo offline com sincronização	Continuidade das atividades sem conexão
Internacionalização (i18n)	Suporte a múltiplos idiomas
Inteligência analítica	Apoio à gestão por indicadores
Integrações externas	Ampliação do ecossistema
Recursos de acessibilidade avançada	Inclusão e conformidade com WCAG

Essas iniciativas dependerão da maturidade do projeto e da disponibilidade de recursos.

5.8 Planejamento por Releases

As funcionalidades podem ser distribuídas por versões.

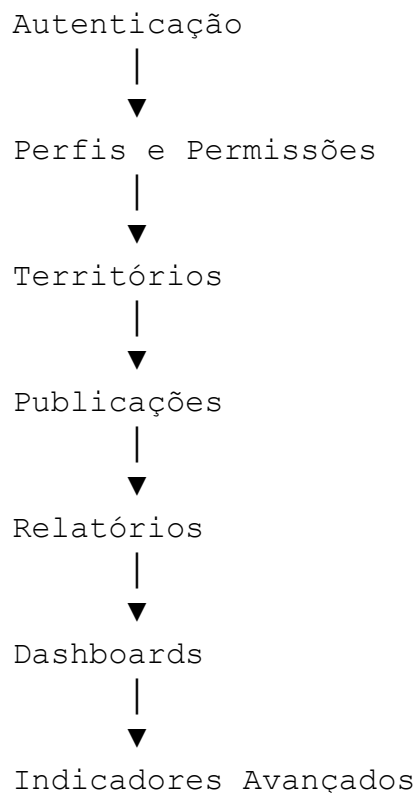
Release Objetivo Principal

- 1.1 Melhorias operacionais e correções
- 1.2 Relatórios e dashboards aprimorados
- 1.3 Funcionalidades administrativas adicionais
- 2.0 Evolução arquitetural e novos recursos estratégicos
- 3.0 Expansão da plataforma e novos serviços

Esse planejamento serve como referência e poderá ser ajustado conforme a evolução do projeto.

5.9 Dependências Funcionais

Algumas funcionalidades dependem da implementação de outras.



O gerenciamento dessas dependências reduz retrabalho e facilita o planejamento das entregas.

5.10 Gestão do Backlog

O roadmap está diretamente relacionado ao backlog do produto.

Cada item deve possuir:

- identificador único;
- descrição;
- prioridade;
- estimativa;
- dependências;
- critérios de aceite;
- status;
- versão prevista.

A gestão estruturada do backlog facilita o acompanhamento da evolução do sistema.

5.11 Critérios para Inclusão de Novas Funcionalidades

Novas funcionalidades devem ser avaliadas considerando:

- necessidade dos usuários;
- alinhamento aos objetivos do projeto;
- impacto na arquitetura;
- custo de implementação;
- benefícios esperados;
- riscos técnicos;
- esforço de manutenção.

Após a análise, a funcionalidade poderá ser incorporada ao roadmap em uma versão futura.

5.12 Revisão do Roadmap

O roadmap não deve ser considerado um documento estático.

Recomenda-se sua revisão:

- ao final de cada release;
- após mudanças significativas de escopo;
- quando novas prioridades forem identificadas;
- diante de alterações tecnológicas relevantes;
- durante o planejamento anual do projeto.

Essa prática mantém o planejamento alinhado à realidade do produto.

5.13 Integração com os Demais Documentos

O roadmap de funcionalidades está relacionado aos seguintes documentos:

Documento	Relação
Documento 01 – Requisitos	Origem das funcionalidades
Documento 03 – Casos de Uso	Fluxos de utilização
Documento 05 – Arquitetura Geral	Impacto arquitetural
Documento 06 – Arquitetura Backend	Implementação das regras de negócio
Documento 07 – Arquitetura Frontend	Evolução da interface
Documento 13 – Estratégia de Testes	Validação das novas funcionalidades

Essa integração garante consistência entre planejamento, implementação e validação.

5.14 Boas Práticas

Para manter um roadmap eficiente, recomenda-se:

- priorizar funcionalidades de maior valor para o usuário;
- manter o backlog organizado;

- revisar prioridades periodicamente;
- documentar dependências entre funcionalidades;
- evitar planejamento excessivamente detalhado para horizontes de longo prazo;
- alinhar as entregas à capacidade da equipe;
- registrar decisões que alterem significativamente o planejamento.

5.15 Conclusão do Capítulo

Este capítulo apresentou o roadmap funcional do GTP, definindo uma visão estratégica para a evolução da plataforma e organizando as funcionalidades previstas em horizontes de curto, médio e longo prazo.

A utilização de um roadmap estruturado facilita o planejamento das releases, promove transparência para as equipes envolvidas e contribui para uma evolução consistente e sustentável do sistema.

6 – Roadmap Tecnológico

6.1 Objetivo

Este capítulo apresenta o **Roadmap Tecnológico** do **Gestor de Territórios e Publicações (GTP)**, estabelecendo a estratégia para evolução das tecnologias, ferramentas, frameworks e infraestrutura que sustentam a plataforma.

O objetivo é garantir que o GTP permaneça atualizado, seguro, escalável e preparado para acompanhar a evolução das boas práticas da Engenharia de Software, DevOps e Arquitetura de Sistemas.

6.2 Visão Estratégica

A evolução tecnológica do GTP é orientada pelos seguintes princípios:

- estabilidade da plataforma;
- adoção de tecnologias consolidadas;
- compatibilidade entre componentes;

- segurança da informação;
- facilidade de manutenção;
- escalabilidade;
- automação de processos;
- redução da dívida técnica (*Technical Debt*).

As atualizações devem ser planejadas de forma gradual para minimizar riscos e evitar interrupções nos serviços.

6.3 Arquitetura Tecnológica Atual

A arquitetura tecnológica da versão atual é composta pelos seguintes componentes:

Camada	Tecnologia
Frontend	React + Vite
Backend	Java 21 + Spring Boot 3
Segurança	Spring Security + JWT
Banco de Dados	PostgreSQL
Migrações	Flyway
ORM	Spring Data JPA / Hibernate
Build Backend	Maven
Build Frontend	Vite
Contêineres	Docker e Docker Compose
Controle de Versão	Git + GitHub
CI/CD	GitHub Actions (ou ferramenta equivalente)

Essa base tecnológica foi escolhida por sua maturidade, amplo suporte da comunidade e aderência às necessidades do projeto.

6.4 Evolução do Backend

O roadmap prevê a atualização contínua do ecossistema Java.

Planejamento:

Tecnologia	Estratégia
Java	Acompanhar versões LTS
Spring Boot	Atualizar para versões estáveis suportadas
Spring Security	Incorporar novos recursos de segurança
Hibernate	Atualizar conforme compatibilidade
Flyway	Evoluir juntamente com o banco de dados
Maven	Atualizações periódicas de dependências

Antes de cada atualização devem ser realizados testes de compatibilidade e regressão.

6.5 Evolução do Frontend

A evolução da interface seguirá práticas modernas de desenvolvimento web.

Objetivos:

- atualização contínua do React;
- atualização do Vite;
- adoção de novos recursos do ECMAScript;
- otimização do desempenho;
- melhoria da acessibilidade;
- expansão do Design System;
- evolução da experiência do usuário (UX).

Também está prevista a ampliação da reutilização de componentes e a padronização visual da aplicação.

6.6 Evolução do Banco de Dados

A estratégia para a camada de persistência contempla:

- atualização do PostgreSQL para versões suportadas;
- revisão periódica de índices;
- otimização de consultas SQL;
- evolução dos scripts Flyway;
- monitoramento de desempenho;
- políticas de retenção de dados;
- preparação para cenários de maior volume de informações.

Mudanças estruturais deverão ser implementadas por meio de migrações versionadas.

6.7 Evolução da Infraestrutura

A infraestrutura será modernizada de forma incremental.

Planejamento:

Área	Evolução Prevista
Docker	Atualizações contínuas
Docker Compose	Revisões conforme evolução da plataforma
Proxy Reverso	Atualizações do Nginx ou solução equivalente
HTTPS	Renovação e automação de certificados
Monitoramento	Ampliação da observabilidade
Backups	Automatização e validação periódica

Essas melhorias buscam aumentar a confiabilidade e reduzir o esforço operacional.

6.8 Observabilidade

A observabilidade será ampliada gradualmente para proporcionar maior visibilidade sobre o comportamento da aplicação.

Áreas previstas:

- métricas de infraestrutura;
- métricas da aplicação;
- centralização de logs;
- rastreamento distribuído (*Distributed Tracing*);
- alertas automáticos;
- dashboards operacionais.

Essa estratégia permitirá identificar problemas de forma proativa e reduzir o tempo de resposta a incidentes.

6.9 Segurança Tecnológica

A evolução da segurança deve acompanhar as melhores práticas do mercado.

Planejamento:

- atualização periódica das dependências;
- correção de vulnerabilidades conhecidas;
- fortalecimento da autenticação e autorização;
- revisão das políticas de acesso;
- ampliação da auditoria;
- automação da análise de vulnerabilidades;
- revisão de configurações de infraestrutura.

As atualizações devem considerar as recomendações da comunidade e os boletins de segurança dos fornecedores das tecnologias utilizadas.

6.10 Automação

O roadmap prevê aumento progressivo da automação.

Exemplos:

- execução automática de testes;
- validação de qualidade do código;
- análise estática;
- geração automática de documentação;
- deploy automatizado;
- backups automáticos;
- monitoramento contínuo;
- geração de relatórios operacionais.

A automação reduz erros humanos e aumenta a eficiência do processo de desenvolvimento e operação.

6.11 Tecnologias Futuras

Dependendo da evolução do projeto, poderão ser avaliadas tecnologias complementares.

Exemplos:

Tecnologia	Possível Aplicação
Kubernetes	Orquestração de contêineres
Redis	Cache distribuído
RabbitMQ ou Apache Kafka	Mensageria assíncrona
OpenTelemetry	Observabilidade e rastreamento
Prometheus	Coleta de métricas
Grafana	Dashboards operacionais
Keycloak	Gerenciamento centralizado de identidade
Elastic Stack	Pesquisa, logs e observabilidade

A adoção dessas tecnologias dependerá das necessidades da plataforma e da maturidade operacional do projeto.

6.12 Gestão da Dívida Técnica

A evolução tecnológica deve incluir ações para controlar a dívida técnica.

Recomenda-se:

- atualização regular das dependências;
- remoção de código obsoleto;
- refatorações planejadas;
- melhoria da cobertura de testes;
- revisão dos padrões arquiteturais;
- eliminação de duplicações;
- monitoramento contínuo da qualidade do código.

Essas práticas contribuem para reduzir custos de manutenção e facilitar futuras evoluções.

6.13 Revisão do Roadmap Tecnológico

O roadmap tecnológico deve ser revisado periodicamente.

Recomenda-se realizar revisões:

- ao final de cada grande release;
- quando houver novas versões LTS das tecnologias principais;
- após mudanças significativas na arquitetura;
- diante de novas exigências de segurança;
- durante o planejamento estratégico anual.

Essa revisão garante que a plataforma permaneça alinhada às melhores práticas do mercado.

6.14 Integração com os Demais Documentos

O roadmap tecnológico está diretamente relacionado aos seguintes documentos:

Documento	Relação
Documento 05 – Arquitetura Geral	Direcionamento arquitetural
Documento 06 – Arquitetura Backend	Evolução tecnológica do backend
Documento 07 – Arquitetura Frontend	Evolução tecnológica do frontend
Documento 10 – Segurança	Atualizações de segurança
Documento 11 – Banco de Dados PostgreSQL	Evolução da persistência
Documento 13 – Estratégia de Testes	Adequação dos testes às novas tecnologias
Documento 14 – Deploy e Infraestrutura	Evolução da infraestrutura e automação

Essa integração assegura consistência entre planejamento tecnológico, arquitetura e implementação.

6.15 Boas Práticas

Para garantir uma evolução tecnológica sustentável, recomenda-se:

- priorizar versões LTS sempre que possível;
- realizar atualizações de forma incremental;
- validar compatibilidade antes das implantações;
- manter documentação técnica atualizada;
- automatizar testes de regressão;
- acompanhar boletins de segurança das tecnologias utilizadas;
- revisar periodicamente a arquitetura;
- monitorar continuamente a dívida técnica.

6.16 Conclusão do Capítulo

Este capítulo apresentou o roadmap tecnológico do GTP, estabelecendo as diretrizes para a evolução das tecnologias utilizadas na plataforma, incluindo

frontend, backend, banco de dados, infraestrutura, observabilidade, segurança e automação.

A adoção de um planejamento tecnológico estruturado contribui para manter o sistema atualizado, reduzir riscos de obsolescência, fortalecer a segurança e garantir que a plataforma permaneça preparada para atender às necessidades futuras.

7 – Planejamento das Próximas Releases

7.1 Objetivo

Este capítulo estabelece a estratégia para o planejamento das próximas releases do **Gestor de Territórios e Publicações (GTP)**, definindo como novas funcionalidades, melhorias e correções serão organizadas, desenvolvidas, validadas e disponibilizadas aos usuários.

O planejamento de releases busca garantir entregas previsíveis, estáveis e alinhadas aos objetivos do projeto, mantendo um equilíbrio entre inovação, qualidade e continuidade operacional.

7.2 Estratégia de Releases

O GTP adota uma estratégia de evolução incremental, na qual cada release representa um conjunto coeso de funcionalidades, melhorias e correções.

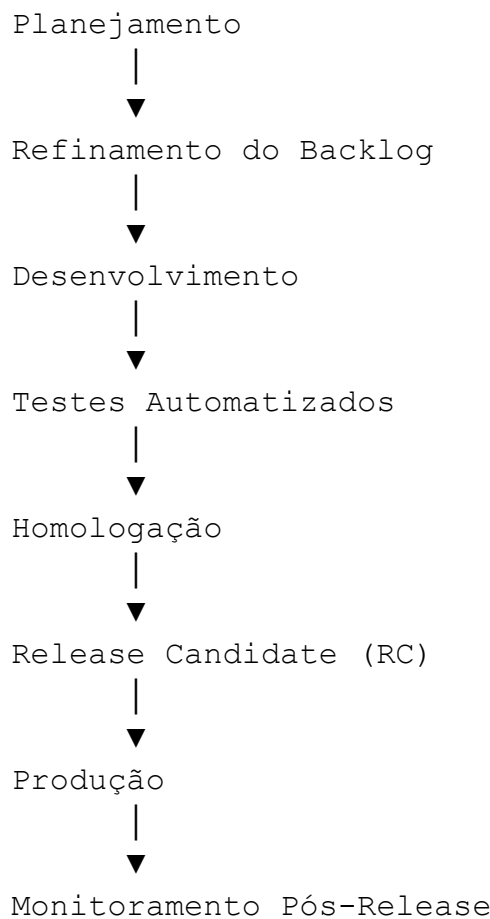
Os princípios adotados são:

- entregas frequentes e controladas;
- priorização por valor de negócio;
- redução de riscos técnicos;
- validação contínua;
- facilidade de rollback;
- documentação completa de cada versão.

Cada release deve possuir escopo claramente definido antes do início do desenvolvimento.

7.3 Ciclo de Vida de uma Release

Cada versão percorre um fluxo padronizado desde o planejamento até sua disponibilização.



Esse fluxo reduz riscos e aumenta a confiabilidade das implantações.

7.4 Tipos de Releases

O projeto utiliza diferentes categorias de releases.

Tipo	Finalidade
Alpha	Desenvolvimento inicial
Beta	Validação funcional
Release Candidate (RC)	Preparação para produção
Stable	Versão oficial para produção
Hotfix	Correções emergenciais

Tipo	Finalidade
Patch	Correções menores
Minor	Inclusão de novas funcionalidades compatíveis
Major	Grandes evoluções e mudanças arquiteturais

A classificação facilita o entendimento do impacto de cada entrega.

7.5 Planejamento das Versões

O roadmap inicial poderá seguir a estrutura abaixo.

Versão Objetivo Principal

- 1.1 Melhorias operacionais e correções
- 1.2 Relatórios avançados e dashboards
- 1.3 Evolução administrativa
- 1.4 Integrações e otimizações
- 2.0 Modernização arquitetural
- 2.1 Expansão funcional
- 3.0 Nova geração da plataforma

Esse planejamento é indicativo e poderá ser ajustado conforme a evolução do projeto.

7.6 Critérios de Entrada (Definition of Ready)

Uma funcionalidade somente poderá entrar em uma release quando atender aos seguintes critérios:

- requisito aprovado;
- análise técnica concluída;
- critérios de aceite definidos;
- dependências identificadas;

- estimativa realizada;
- riscos avaliados;
- documentação inicial disponível.

Esses critérios reduzem incertezas durante o desenvolvimento.

7.7 Critérios de Saída (Definition of Done)

Uma funcionalidade será considerada concluída quando atender aos seguintes requisitos:

- implementação finalizada;
- revisão de código realizada;
- testes automatizados aprovados;
- testes funcionais concluídos;
- documentação atualizada;
- migrações executadas com sucesso;
- aprovação da homologação;
- integração com os demais módulos validada.

Somente funcionalidades que atendam a todos esses critérios poderão compor uma release estável.

7.8 Gestão de Dependências

O planejamento deve considerar dependências técnicas e funcionais entre módulos.

Exemplo de sequência lógica:

Autenticação



Perfis e Permissões



Usuários



Territórios



Publicações



Relatórios



Dashboards

A gestão adequada dessas dependências evita retrabalho e reduz riscos de integração.

7.9 Gestão de Riscos

Antes da publicação de cada release, devem ser avaliados os principais riscos.

Risco	Mitigação
Falhas de integração	Testes integrados
Regressões	Testes automatizados
Problemas de desempenho	Testes de carga
Vulnerabilidades	Análise de segurança
Falhas de infraestrutura	Ambiente de homologação semelhante ao de produção
Erros de implantação	Pipeline automatizado e plano de rollback

O gerenciamento preventivo dos riscos aumenta a estabilidade das entregas.

7.10 Processo de Homologação

Antes da publicação em produção, toda release deve passar por homologação.

As etapas incluem:

- validação funcional;
- testes de integração;
- testes de regressão;
- validação de segurança;
- verificação de desempenho;
- revisão da documentação;
- aprovação dos responsáveis.

Somente após a conclusão dessas etapas a release poderá ser promovida para produção.

7.11 Publicação da Release

A disponibilização de uma nova versão deve seguir um procedimento padronizado.

Etapas recomendadas:

1. geração da versão final;
2. criação da tag no Git;
3. execução do pipeline de CI/CD;
4. implantação em produção;
5. validação pós-deploy;
6. monitoramento intensivo;
7. atualização do changelog;
8. comunicação aos usuários.

Esse processo garante rastreabilidade e controle das implantações.

7.12 Plano de Rollback

Toda release deve possuir um plano de reversão.

O plano deve contemplar:

- critérios para rollback;
- responsáveis pela decisão;
- procedimentos técnicos;
- restauração do banco de dados (quando necessário);
- restauração da versão anterior;
- validação dos serviços;
- comunicação aos usuários.

A existência de um plano de rollback reduz o impacto de falhas críticas.

7.13 Indicadores das Releases

A qualidade das releases pode ser acompanhada por meio de indicadores.

Indicador	Objetivo
Frequência de releases	Medir capacidade de entrega
Taxa de sucesso das implantações	Avaliar estabilidade
Número de rollback	Identificar problemas nas publicações
Incidentes pós-release	Medir qualidade das entregas
Tempo médio de implantação	Avaliar eficiência operacional
Tempo de recuperação	Medir capacidade de resposta

Esses indicadores apoiam a melhoria contínua do processo de entrega.

7.14 Integração com CI/CD

O planejamento das releases está diretamente integrado ao processo de integração e entrega contínuas.

Cada release deve utilizar pipelines automatizados para:

- compilação;
- execução dos testes;
- análise de qualidade do código;
- geração de artefatos;
- implantação;
- monitoramento inicial.

Essa automação reduz falhas humanas e aumenta a previsibilidade das implantações.

7.15 Integração com os Demais Documentos

O planejamento das releases está relacionado aos seguintes documentos:

Documento	Relação
Documento 01 – Especificação de Requisitos	Origem das funcionalidades
Documento 05 – Arquitetura Geral	Impacto arquitetural
Documento 13 – Estratégia de Testes	Validação das releases
Documento 14 – Deploy e Infraestrutura	Processo de implantação
Documento 15 – Manual do Desenvolvedor	Fluxo de desenvolvimento
Documento 19 – Guia de Operação e Suporte	Operação pós-release

Essa integração assegura que cada versão seja planejada, implementada, validada e disponibilizada de forma consistente.

7.16 Boas Práticas

Para garantir releases previsíveis e de alta qualidade, recomenda-se:

- manter ciclos de entrega regulares;
- evitar alterações de escopo durante o desenvolvimento da release;
- automatizar testes e implantações;

- revisar continuamente o backlog;
- documentar todas as mudanças;
- comunicar as equipes envolvidas;
- monitorar os resultados após cada implantação;
- utilizar retrospectivas para aperfeiçoar o processo.

7.17 Conclusão do Capítulo

Este capítulo apresentou o planejamento das próximas releases do GTP, definindo uma estratégia estruturada para organização das entregas, gestão de riscos, homologação, publicação e monitoramento das versões.

A adoção de um processo padronizado de releases fortalece a qualidade do software, reduz riscos operacionais e contribui para uma evolução contínua, previsível e sustentável da plataforma.

8 – Gestão do Ciclo de Vida

8.1 Objetivo

Este capítulo estabelece as diretrizes para a **Gestão do Ciclo de Vida** do **Gestor de Territórios e Publicações (GTP)**, definindo como o produto será planejado, desenvolvido, mantido, evoluído e, quando necessário, descontinuado.

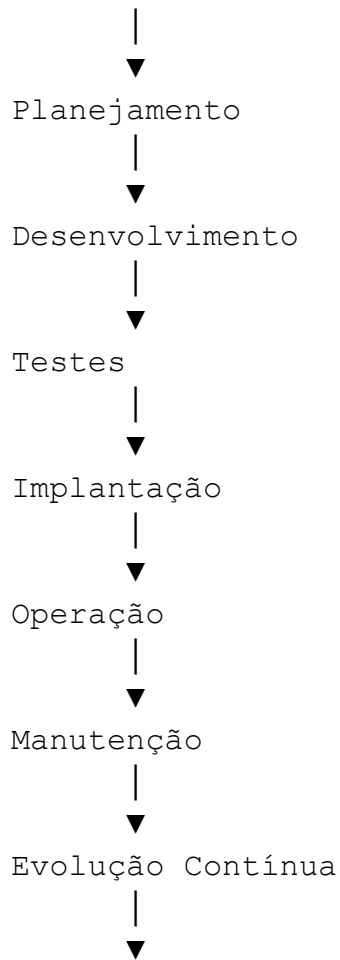
O gerenciamento adequado do ciclo de vida garante que a plataforma permaneça segura, confiável, atualizada e preparada para atender às necessidades dos usuários ao longo do tempo.

8.2 Conceito de Ciclo de Vida

O ciclo de vida do software representa todas as fases pelas quais o GTP passa desde sua concepção até uma eventual substituição ou encerramento.

As principais etapas são:

Concepção



Descontinuação (quando aplicável)

Cada fase possui objetivos, responsabilidades e entregáveis específicos.

8.3 Fases do Ciclo de Vida

Concepção

Nesta fase são definidos:

- objetivos do projeto;
- escopo inicial;
- requisitos de alto nível;
- público-alvo;
- viabilidade técnica.

Planejamento

São realizadas:

- definição da arquitetura;
- planejamento das releases;
- organização do backlog;
- estimativas;
- cronograma.

Desenvolvimento

Compreende:

- implementação das funcionalidades;
- criação dos testes;
- revisão de código;
- documentação técnica;
- integração contínua.

Testes

Inclui:

- testes unitários;
- testes de integração;
- testes funcionais;
- testes de regressão;
- validação de segurança;
- homologação.

Implantação

Abrange:

- geração da release;

- execução do pipeline de CI/CD;
- deploy;
- validação em produção;
- monitoramento inicial.

Operação

Durante esta fase são executadas:

- monitoramento contínuo;
- suporte aos usuários;
- acompanhamento dos indicadores;
- gestão de incidentes;
- manutenção operacional.

Manutenção

São realizadas:

- correções;
- atualizações;
- otimizações;
- melhorias de segurança;
- refatorações.

Evolução

A plataforma passa por:

- novas funcionalidades;
- evolução tecnológica;
- melhorias arquiteturais;
- modernização da infraestrutura.

8.4 Tipos de Manutenção

O GTP adota os quatro tipos clássicos de manutenção de software.

Tipo	Objetivo
Corretiva	Correção de defeitos
Adaptativa	Adequação a novos ambientes
Evolutiva	Inclusão de novas funcionalidades
Preventiva	Redução de riscos futuros

Cada manutenção deve ser registrada no histórico de versões.

8.5 Política de Suporte às Versões

As versões do sistema seguem uma política de suporte baseada em sua criticidade.

Tipo de Versão	Política
Stable	Suporte completo
LTS (quando adotada)	Suporte estendido
Beta	Suporte limitado aos testes
Alpha	Uso exclusivo para desenvolvimento
Hotfix	Correções emergenciais

A definição de uma versão LTS poderá ser adotada futuramente para ambientes que demandem maior estabilidade.

8.6 Gestão da Obsolescência Tecnológica

A evolução tecnológica exige acompanhamento constante das dependências utilizadas.

Recomenda-se:

- monitorar versões dos frameworks;

- acompanhar o ciclo de suporte das tecnologias;
- atualizar dependências regularmente;
- remover bibliotecas obsoletas;
- revisar periodicamente a arquitetura.

Essa prática reduz riscos de segurança e facilita futuras evoluções.

8.7 Gestão da Dívida Técnica

A dívida técnica representa decisões que aceleram o desenvolvimento no curto prazo, mas aumentam o custo de manutenção no futuro.

Para controlá-la, recomenda-se:

- refatorações periódicas;
- revisão de código;
- melhoria da cobertura de testes;
- atualização da documentação;
- eliminação de duplicações;
- simplificação da arquitetura.

A dívida técnica deve ser monitorada continuamente e tratada como parte do planejamento das releases.

8.8 Evolução Contínua

A evolução do GTP é baseada no princípio de melhoria contínua.

As principais ações incluem:

- revisão do roadmap;
- análise do feedback dos usuários;
- avaliação de indicadores;
- incorporação de novas tecnologias;
- otimização da experiência do usuário;
- aprimoramento dos processos internos.

Essa abordagem mantém o produto alinhado às necessidades do negócio e às boas práticas do mercado.

8.9 Gestão de Funcionalidades Descontinuadas

Quando uma funcionalidade precisar ser removida, recomenda-se seguir um processo controlado.

Etapas sugeridas:

1. identificação da necessidade;
2. avaliação dos impactos;
3. comunicação aos usuários;
4. definição do período de transição;
5. atualização da documentação;
6. remoção na versão planejada.

Sempre que possível, deve ser oferecida uma alternativa antes da descontinuação definitiva.

8.10 Gestão de Dependências

O ciclo de vida também contempla o gerenciamento das dependências técnicas.

Devem ser monitorados:

- frameworks;
- bibliotecas;
- banco de dados;
- ferramentas de build;
- ferramentas de segurança;
- infraestrutura;
- sistemas operacionais suportados.

Atualizações devem ser planejadas para evitar incompatibilidades.

8.11 Indicadores do Ciclo de Vida

A evolução do produto pode ser acompanhada por indicadores.

Indicador	Objetivo
Frequência de releases	Evolução contínua
Tempo médio de correção	Eficiência da manutenção
Cobertura de testes	Qualidade do software
Débito técnico estimado	Sustentabilidade da arquitetura
Taxa de defeitos	Estabilidade da plataforma
Disponibilidade do sistema	Continuidade operacional

Esses indicadores subsidiam decisões estratégicas sobre a evolução do produto.

8.12 Papéis e Responsabilidades

A gestão do ciclo de vida envolve diferentes perfis.

Papel	Responsabilidade
Product Owner	Definição das prioridades
Arquiteto de Software	Evolução arquitetural
Desenvolvedores	Implementação
DevOps	Infraestrutura e deploy
QA	Garantia da qualidade
Administradores	Operação do sistema
Suporte	Atendimento aos usuários

A colaboração entre essas equipes garante a continuidade e a evolução sustentável do GTP.

8.13 Integração com os Demais Documentos

A gestão do ciclo de vida relaciona-se diretamente aos seguintes documentos:

Documento	Relação
Documento 01 – Especificação de Requisitos	Evolução das funcionalidades
Documento 05 – Arquitetura Geral	Evolução arquitetural
Documento 13 – Estratégia de Testes	Garantia da qualidade
Documento 14 – Deploy e Infraestrutura	Implantação e operação
Documento 15 – Manual do Desenvolvedor	Processo de desenvolvimento
Documento 19 – Guia de Operação e Suporte	Operação e manutenção
Documento 21 – Governança e Gestão do Projeto	Processos de gestão e tomada de decisão

Essa integração assegura consistência entre planejamento, desenvolvimento, operação e evolução da plataforma.

8.14 Boas Práticas

Para garantir um ciclo de vida eficiente, recomenda-se:

- manter documentação sempre atualizada;
- revisar periodicamente o roadmap;
- controlar a dívida técnica;
- automatizar testes e implantações;
- monitorar continuamente indicadores de qualidade;
- atualizar dependências regularmente;
- documentar todas as mudanças relevantes;
- planejar a descontinuação de funcionalidades de forma transparente.

8.15 Conclusão do Capítulo

Este capítulo apresentou as diretrizes para a gestão do ciclo de vida do GTP, definindo processos para desenvolvimento, manutenção, evolução, suporte e eventual descontinuação de funcionalidades.

A adoção de um ciclo de vida estruturado fortalece a sustentabilidade do projeto, reduz riscos de obsolescência tecnológica, melhora a qualidade do software e assegura que a plataforma continue evoluindo de forma organizada, previsível e alinhada às necessidades dos usuários.

9 – Boas Práticas de Versionamento

9.1 Objetivo

Este capítulo estabelece as diretrizes de **versionamento** adotadas pelo **Gestor de Territórios e Publicações (GTP)**, definindo padrões para identificação das versões, organização do repositório Git, criação de branches, utilização de tags, documentação das alterações e rastreabilidade das releases.

A adoção de práticas padronizadas de versionamento contribui para a organização do projeto, facilita a colaboração entre desenvolvedores e aumenta a confiabilidade do processo de entrega contínua.

9.2 Princípios do Versionamento

O versionamento do GTP baseia-se nos seguintes princípios:

- simplicidade;
- consistência;
- rastreabilidade;
- previsibilidade;
- compatibilidade;
- transparência;
- documentação contínua.

Cada versão deve representar um conjunto consistente de alterações validadas e documentadas.

9.3 Versionamento Semântico (Semantic Versioning)

O GTP adota o padrão **Semantic Versioning (SemVer)**.

A estrutura das versões segue o formato:

MAJOR.MINOR.PATCH

Onde:

Elemento Descrição

MAJOR Alterações incompatíveis ou grandes mudanças arquiteturais

MINOR Inclusão de novas funcionalidades compatíveis

PATCH Correções de defeitos, ajustes e pequenas melhorias

Exemplos

Versão Significado

1.0.0 Primeira versão oficial

1.1.0 Inclusão de novas funcionalidades

1.1.1 Correção de defeitos

1.2.0 Evolução funcional

2.0.0 Nova geração da plataforma

9.4 Identificação das Releases

Cada release deverá conter informações padronizadas.

Campo	Conteúdo
-------	----------

Número da versão	Ex.: 1.2.0
------------------	------------

Data da publicação	DD/MM/AAAA
--------------------	------------

Tipo da release	Alpha, Beta, RC, Stable ou Hotfix
-----------------	-----------------------------------

Ambiente	Desenvolvimento, Homologação ou Produção
----------	--

Campo	Conteúdo
Responsável	Equipe de Desenvolvimento
Changelog	Lista oficial das alterações

Essa padronização facilita auditorias e consultas futuras.

9.5 Estratégia de Branches

O controle de versões utiliza o Git com uma estratégia de branches baseada em boas práticas de desenvolvimento colaborativo.

Branches permanentes

Branch Finalidade

main	Código oficial em produção
develop	Integração contínua das funcionalidades

Branches temporárias

Prefixo Finalidade

feature/	Desenvolvimento de novas funcionalidades
bugfix/	Correção de defeitos não críticos
hotfix/	Correções emergenciais em produção
release/	Preparação de uma nova versão
refactor/	Melhorias estruturais sem alteração funcional
docs/	Atualizações da documentação
test/	Experimentos e validações temporárias

Exemplos:

```
feature/cadastro-publicadores
feature/dashboard-gerencial
bugfix/login-jwt
hotfix/correcao-token-expirado
```

```
release/1.2.0  
refactor/service-territorios  
docs/manual-administrador
```

Essa convenção facilita a organização do histórico do repositório.

9.6 Convenção para Tags

Toda versão publicada deve possuir uma **tag Git** correspondente.

Modelo:

```
v1.0.0  
v1.1.0  
v1.1.1  
v2.0.0
```

As tags devem apontar para o commit oficial da release publicada.

9.7 Convenção para Commits

Os commits devem seguir um padrão descritivo e consistente.

Recomenda-se utilizar o padrão **Conventional Commits**.

Prefixo	Utilização
feat:	Nova funcionalidade
fix:	Correção de defeito
docs:	Documentação
style:	Ajustes de formatação
refactor:	Refatoração
test:	Inclusão ou alteração de testes
perf:	Melhorias de desempenho
build:	Alterações de build

Prefixo	Utilização
ci:	Alterações em CI/CD
chore:	Tarefas administrativas

Exemplos

```
feat: implementar gerenciamento de territórios  
fix: corrigir validação de autenticação JWT  
docs: atualizar Manual do Desenvolvedor  
refactor: reorganizar camada de serviços  
perf: otimizar consultas de relatórios
```

9.8 Integração com GitHub

O fluxo de trabalho no GitHub deve contemplar:

- criação de Pull Requests;
- revisão obrigatória de código;
- execução automática dos pipelines de CI;
- validação dos testes;
- aprovação antes do merge;
- vinculação aos requisitos e às issues correspondentes.

Esse processo aumenta a qualidade das entregas e reduz riscos de regressão.

9.9 Controle do Changelog

Cada release deve atualizar o **Documento 20 – Histórico de Versões e Roadmap** e o arquivo CHANGELOG.md do repositório.

O registro deve incluir:

- funcionalidades adicionadas;
- alterações realizadas;
- correções;

- melhorias de desempenho;
- mudanças de segurança;
- alterações de infraestrutura;
- atualizações da documentação.

Essa prática garante sincronização entre código e documentação.

9.10 Rastreabilidade

Toda alteração deve ser rastreável.

Sempre que possível, cada funcionalidade deve estar vinculada a:

- requisito funcional;
- requisito não funcional;
- caso de uso;
- issue;
- Pull Request;
- commit;
- release;
- documentação correspondente.

Essa rastreabilidade facilita auditorias e manutenção.

9.11 Versionamento da Documentação

Além do código-fonte, toda documentação do projeto deve ser versionada.

Devem ser controlados:

- documentos técnicos;
- manuais;
- diagramas;
- modelos de banco de dados;
- APIs;

- arquitetura;
- políticas de segurança.

Cada atualização significativa deve indicar:

- versão;
- data;
- responsável;
- resumo das alterações.

9.12 Versionamento do Banco de Dados

As alterações estruturais no banco de dados devem utilizar migrações versionadas.

No GTP, as migrações são controladas por meio do **Flyway**, garantindo:

- histórico das alterações;
- execução ordenada;
- repetibilidade;
- controle de ambiente;
- rollback planejado (quando aplicável).

Nenhuma alteração estrutural deve ser realizada diretamente em produção sem uma migração correspondente.

9.13 Versionamento da API

A API REST deve manter compatibilidade sempre que possível.

Boas práticas:

- evitar mudanças incompatíveis em versões menores;
- documentar alterações na API;
- versionar endpoints quando necessário;
- manter documentação atualizada (OpenAPI/Swagger);
- comunicar mudanças que impactem consumidores da API.

9.14 Auditoria das Releases

Cada release deve possuir um registro contendo:

Informação	Descrição
Número da versão	Identificação da release
Data	Publicação
Responsáveis	Equipe envolvida
Commits	Lista de alterações
Pull Requests	Revisões realizadas
Pipeline	Resultado da implantação
Changelog	Alterações documentadas

Esses registros fortalecem a governança do projeto.

9.15 Boas Práticas

Para manter um versionamento consistente, recomenda-se:

- seguir rigorosamente o Semantic Versioning;
- criar tags para todas as releases oficiais;
- utilizar commits padronizados;
- revisar código antes do merge;
- manter o changelog atualizado;
- documentar alterações arquiteturais relevantes;
- versionar banco de dados e documentação;
- manter rastreabilidade entre requisitos, código e releases;
- evitar alterações diretas na branch principal (main);
- automatizar validações por meio do pipeline de CI/CD.

9.16 Conclusão do Capítulo

Este capítulo apresentou as boas práticas de versionamento adotadas pelo GTP, estabelecendo padrões para gerenciamento de versões, branches, commits, tags, documentação e rastreabilidade.

A adoção dessas práticas fortalece a governança do projeto, facilita o trabalho colaborativo, melhora a qualidade das entregas e garante um histórico consistente da evolução da plataforma.

10 – Conclusão

10.1 Encerramento do Documento

Este documento consolidou a estratégia de evolução do **Gestor de Territórios e Publicações (GTP)**, estabelecendo as diretrizes para o gerenciamento do histórico de versões, planejamento de releases, evolução arquitetural, roadmap funcional, roadmap tecnológico e gestão do ciclo de vida da plataforma.

Mais do que um registro histórico, este documento representa um instrumento de governança que orienta a evolução do sistema de forma estruturada, previsível e alinhada às boas práticas da Engenharia de Software.

10.2 Síntese dos Conteúdos

Ao longo dos capítulos foram apresentados os principais elementos necessários para controlar a evolução do GTP.

Os temas abordados incluem:

- histórico oficial das versões;
- changelog estruturado;
- evolução arquitetural;
- roadmap funcional;
- roadmap tecnológico;
- planejamento das releases;
- gestão do ciclo de vida do software;
- boas práticas de versionamento;

- diretrizes para manutenção da documentação.

Em conjunto, esses elementos proporcionam uma visão completa do processo de evolução da plataforma.

10.3 Governança da Evolução

A evolução do GTP deverá seguir princípios claros de governança, garantindo que todas as mudanças sejam devidamente planejadas, implementadas, testadas, documentadas e monitoradas.

Os principais pilares dessa governança são:

- planejamento estratégico;
- rastreabilidade das alterações;
- documentação contínua;
- versionamento padronizado;
- automação dos processos;
- gestão de riscos;
- melhoria contínua.

Essa abordagem contribui para a estabilidade do produto e para a sustentabilidade do projeto a longo prazo.

10.4 Evolução Contínua

O GTP foi concebido como uma plataforma em constante evolução.

Cada nova versão deverá representar um avanço em pelo menos um dos seguintes aspectos:

- funcionalidades;
- experiência do usuário;
- desempenho;
- segurança;
- arquitetura;
- infraestrutura;

- qualidade do código;
- observabilidade;
- documentação.

A evolução contínua permite que a plataforma acompanhe novas necessidades operacionais, mudanças tecnológicas e boas práticas da indústria.

10.5 Integração entre Planejamento e Desenvolvimento

O roadmap definido neste documento estabelece uma ligação direta entre o planejamento estratégico e a implementação técnica.

Esse alinhamento proporciona:

- maior previsibilidade das entregas;
- melhor utilização dos recursos disponíveis;
- redução de retrabalho;
- organização das prioridades;
- evolução incremental da plataforma;
- maior transparência para todos os envolvidos.

O planejamento poderá ser revisado periodicamente para refletir novas demandas e oportunidades.

10.6 Importância do Histórico de Versões

A manutenção de um histórico detalhado das versões oferece diversos benefícios, entre eles:

- rastreabilidade das mudanças;
- apoio às auditorias;
- documentação das decisões técnicas;
- registro da evolução funcional;
- acompanhamento da evolução tecnológica;
- suporte às atividades de manutenção;

- preservação do conhecimento do projeto.

O histórico de versões torna-se, assim, um patrimônio técnico da plataforma.

10.7 Compromisso com a Qualidade

Todas as futuras evoluções do GTP deverão manter o compromisso com:

- qualidade do software;
- segurança da informação;
- confiabilidade operacional;
- desempenho;
- acessibilidade;
- usabilidade;
- escalabilidade;
- manutenibilidade.

Esses princípios orientam tanto as decisões técnicas quanto o planejamento das releases.

10.8 Perspectivas Futuras

A evolução do GTP poderá contemplar, conforme as necessidades do projeto e a disponibilidade de recursos, iniciativas como:

- expansão dos módulos funcionais;
- novos painéis analíticos;
- melhorias na experiência do usuário;
- recursos avançados de acessibilidade;
- integração com serviços externos;
- Progressive Web App (PWA);
- aplicativo móvel;
- observabilidade avançada;
- maior automação dos processos de desenvolvimento e operação;

- ampliação dos recursos de segurança.

Essas possibilidades representam oportunidades de crescimento da plataforma e deverão ser avaliadas de acordo com os objetivos estratégicos do projeto.

10.9 Responsabilidade pela Atualização

A atualização deste documento deverá ocorrer sempre que houver:

- publicação de uma nova release;
- alteração significativa na arquitetura;
- inclusão ou remoção de funcionalidades relevantes;
- mudanças no roadmap;
- atualização das tecnologias principais;
- revisão das políticas de versionamento.

A responsabilidade pela manutenção do documento é da equipe de desenvolvimento, com apoio dos responsáveis pela arquitetura, infraestrutura e gestão do produto.

Considerações Finais

O **Documento 20 – Histórico de Versões e Roadmap** encerra o conjunto de documentos de governança técnica do **Gestor de Territórios e Publicações (GTP)**, consolidando uma visão abrangente sobre a evolução planejada da plataforma.

Sua utilização como referência oficial permitirá que todas as futuras versões sejam desenvolvidas de maneira organizada, transparente e alinhada às melhores práticas da Engenharia de Software, promovendo consistência entre requisitos, arquitetura, implementação, testes, infraestrutura e operação.

Ao integrar histórico, planejamento e estratégia de evolução em um único documento, o GTP estabelece uma base sólida para seu crescimento sustentável, reduzindo riscos, preservando o conhecimento do projeto e facilitando a colaboração entre desenvolvedores, administradores e demais envolvidos.